

An exact algorithm for correlation clustering on labelled items

Bang Ye Wu*, Li-Hsuan Chen, Chen-Wan Lin and Chun-Chieh Wang
National Chung Cheng University, ChiaYi, Taiwan 621, R.O.C.

Abstract

Motivated by utilizing both the graph clustering and other clustering methods, we introduces a new variant of the correlation clustering problem, in which each item has one or more label indicating which clusters it should belong to. We show that the naive exhaustive search algorithm can be improved by using designed reduction rules in a tree-search algorithm. We also design a lower bound function to speed-up the practical performance. The experimental results show that the improvement is very significant.

Keywords. algorithm, graph clustering, time complexity, parameterized algorithm, NP-hard.

1 Introduction

Graph-based data clustering is widely used in many fields, which is the task of grouping data points based on their similarities. The *correlation clustering* is a graph theoretic approach to clustering [2, 23]. For a given graph which represents the similarity relationship between items, the goal is to partition the items into several *clusters* so as to minimize the total *conflict*, in which the conflict of an item (vertex) is the number of its non-neighbors in the same cluster and neighbors in the different cluster. In the literature, the problem is also be formulated as a graph modification problem known as *cluster graph editing* [23]. A *cluster graph* is a graph composed of disjoint maximal cliques. The cluster graph editing problem asks for a minimum number of edge insertions and deletions to modify the input graph into a cluster graph. As a variant, for a fixed integer $p \geq 2$, the *p-cluster graph editing*, or *p-correlation clustering*, problem is to modify the input into exactly p disjoint cliques [23].

In this paper, we study a variant of the *p-correlation clustering*, in which, in addition to the similarity graph, each vertex is given one or

more label indicating the cluster it can be clustered into. More formally, we are given a graph $G = (V, E)$ and a nonempty label set $L(v)$ for each $v \in V$, in which $L(v) \subset \{1..p\}$. A *p-partition* $\pi = (V_1, V_2, \dots, V_p)$ of V is *feasible* if $i \in L(v)$ for all $v \in V_i$ and $1 \leq i \leq p$. For $u, v \in V$, (u, v) is a *conflict* if the two vertices are in the same cluster but $(u, v) \notin E$ or they are in different clusters but $(u, v) \in E$. Let $C_\pi(v)$ denote the set of vertices conflicting with v in π and $c_\pi(v) = |C_\pi(v)|$ be the *conflict number* of v . Our problem is formally defined as follows in the decision version.

p-CORRELATION CLUSTERING FOR LABELLED ITEMS (p-CCLI)

INSTANCE: A graph $G = (V, E)$, a label set $L(v)$ for each $v \in V$ and a nonnegative integers k .

QUESTION: Is there a feasible *p-partition* π of V with the total conflict number $\sum_{v \in V} c_\pi(v) \leq k$?

The optimization version looks for a feasible *p-partition* with minimum total conflict. The original *p-correlation clustering* problem is equivalent to the case that $L(v) = \{1..p\}$ for each v .

Motivation. The original correlation clustering classifies items only according to the similarity relationship, i.e., similar items should be clustered into the same cluster. However, the correlation clustering problem is NP-hard, and finding an exact solution is very time consuming. On the other hand, we may usually have some knowledge about how these items should be clustered. For example, in the traditional feature-based clustering, there are several predefined categories and each category is associated with several features. Each item will be classified into the category fitting the maximum number of its features. However, there may be more than one category fitting the same number of features of an item, or the difference is not significant. Therefore an item may have more than one label. As a method utilizing both the features and similarities, we may first label the items by some feature-based method and then find the clustering by solving the *p-CCLI* problem. Take

*Corresponding author: bangye@ccu.edu.tw

the problem of categorizing scientific articles as an example. We may label the articles according to the keywords and define the similarity relationship by their citations. That is, the p -CCLI problem provide us an approach to combine the correlation clustering with some other clustering method. More importantly, if the labelling is reliable, there will be not too many multi-labelled items, and the computational complexity of the correlation clustering can be significantly reduced, as shown in this paper.

Previous results. Shamir et al. [23] showed that CLUSTER EDITING is NP-hard. They also showed that p -CLUSTER EDITING is NP-hard for any $p \geq 2$. Therefore the p -CCLI problem studied in this paper is also NP-hard since it includes p -CLUSTER EDITING as a special case. The parameterized version of Cluster Editing, and variants of it, were studied intensively [3, 4, 7, 9, 10, 16, 17, 18]. A variant with vertex (rather than edge) deletions was considered in [20], and another variant in which overlapping clusters are allowed was studied in [12].

The correlation clustering problem (on complete signed graphs) was formulated and studied in [2], in which the authors presented a PTAS for the maximization version and a constant factor approximation algorithm for the minimization version. Ailon et al. [1] showed some approximation results for the minimization version, including the unweighted case and the weighted case with some interesting assumptions on weights. Giotis and Guruswami showed that both the minimization and the maximization versions of the p -Correlation Clustering problem admit PTAS for the unweighted case [15]. The maximization 2-Correlation Clustering problem is also known as BALANCED SUBGRAPH which name comes from the application in social network analysis [19, 24].

For p -CLUSTER EDITING, a kernel with $(p + 2)k + p$ vertices was given by Guo [18]. A variant such that the conflict number of each vertex must be bounded by a parameter was studied in [21]. The problem of finding a 2-clustering minimizing the maximum conflict number has been shown NP-hard [8]. Very recently, Fomin et al. gave a parameterized algorithm with time complexity $O(2^{O(\sqrt{pk})} + n^2)$, where n is the number of vertices [14].

Contributions. In this paper, we design an exact algorithm for p -CCLI with time complexity $O(2^{B-n} + n^2)$, in which B is the total label length

and n is the number of vertices. The complexity is exponential in B but polynomial in n . As pointed out in the motivation, the main goal of defining the problem is for the case that the total label length is limited. It is also the philosophy of developing parameterized algorithms. So it can be also thought of as, however natural, a parameterized algorithm if taking B as a parameter. As shown in Section 2, a naive algorithm solve the problem in $O(n^2 \cdot 2^{B-n})$ time. The main contribution of this work is to illustrate how to use the reduction rules and tree-search algorithm to speed up an exhaustive search algorithm. Another contribution is about the practical performance. The practical performance of a tree-search algorithm can usually significantly improved by designing a good lower bound function to avoid some search, i.e., the branch-and-bound strategy. In this paper, we design a lower bound and, by the experimental results on random data, we show that the effect of the lower bound. In our experiments, the algorithm can solve the cases of $B - n = 100$ and $n = 500$ very efficiently.

The paper is organized as follows. In section 2, define some notation, introduce the basic technique, and analyze the naive algorithm. In Section 3, we show the reduction rules and analyze the time complexity of the search-tree algorithm. The experimental results are shown in Section 4, and finally we give some conclusion remarks in Section 5.

2 preliminaries

In this section, we first define the notation used in this paper. Next, we explain the basic technique for speed-up search-tree algorithm by reduction rules, and then we show the time complexity of the naive algorithm for p -CCLI.

2.1 Notation and definitions

In this paper, we consider only undirected graphs contain no self-loops and multiple edges. For a graph G , $V(G)$ and $E(G)$ denote the vertex and edge sets, respectively. For a vertex subset U , the subgraph of G induced by U is denoted by $G[U]$. In this work, $G = (V, E)$ is the input graph and $n = |V|$. As defined in the introduction, an instance also includes a label set $L(v)$ for each $v \in V$. Let $\lambda(v) = |L(v)|$ be the label size and $B = \sum_{v \in V} \lambda(v)$ the total label size. For each $v \in V$ let $N(v)$ denote the open-neighbourhood

of v , i.e., the set of vertices adjacent to v . The degree of a vertex v is denoted by $d(v) = |N(v)|$. For a vertex subset U , let $d(v, U) = |N(v) \cap U|$ denote the number of neighbours of v in U . The set-minus is denoted by “ $\setminus$ ”, i.e., for two sets S and S' , $S \setminus S' = \{e \mid e \in S \text{ and } e \notin S'\}$.

In this paper we focus on p -clusterings, in which p is a fixed integer at least two. A p -partition $\pi = (V_1, V_2, \dots, V_p)$ of V is a collection of disjoint subsets of V , such that $V_i \cap V_j = \emptyset$ for $i \neq j$ and $V_1 \cup V_2 \cup \dots \cup V_p = V$. In this work we assume $V_i \neq \emptyset$, where $1 \leq i \leq p$. We say that $V_1, V_2, \dots, V_p$ are the p clusters corresponding to π . The conflict set $C_\pi(v)$ and the conflict number $c_\pi(v)$ have been defined in the previous section. Clearly $u \in C_\pi(v)$ if and only if $v \in C_\pi(u)$. For $S \subset V$, let $C_\pi(v, S) = C_\pi(v) \cap S$ and $c_\pi(v, S) = |C_\pi(v, S)|$. For disjoint S_1 and S_2 , $c_\pi(S_1, S_2) = \sum_{v \in S_1} c_\pi(v, S_2)$. Note that $c_\pi(S_1, S_2) = c_\pi(S_2, S_1)$. When there is no confusion, we shall omit the subscript and simply use $c(\cdot)$ instead of $c_\pi(\cdot)$.

2.2 Basic techniques

For a problem with instance size n , if there are $g(n)$ possible solutions and each takes $O(f(n))$ time to construct and evaluate, a naive exhaustive search algorithm takes $O(g(n) \cdot f(n))$ time. The tree-search algorithm is a recursive algorithm which explores all the possible solutions by a tree-like structure. In each recursive call, we make some decision and divide the possible solutions into two or more subclasses (branching). Although basically a tree-search algorithm also generates all possible solutions, we can usually improve the time complexity if we can reduce the instance size in each recursive call. For example, the possible solutions are all the subsets of a set with cardinality n . Each subset with cardinality k takes $O(k^2)$ time to evaluate. The complexity of the naive algorithm is

$$\Theta\left(\sum_{k=1}^n \binom{n}{k} k^2\right) = \Theta(n^2 \cdot 2^n).$$

Note that this bound is not over estimated since $\binom{n}{n/2} (n/2)^2 \in \Theta(n^2 2^n)$. Suppose that we can design a search-tree algorithm which makes the branches by simply picking or discarding an element in each recursive call. The algorithm also generates all the 2^n subsets. However, if we can design a reduction rule such that in each recursive call the processing time is $O(f(k))$, in which f is any polynomial function dependent only on the current instance size k but not one the original instance size n , the time complexity will be

formulated by

$$T(k) = 2T(k-1) + f(k).$$

Solving the recurrence equation, we have that $T(n) = O(2^n)$. That is, we eliminate the polynomial factor in the time complexity. This technique is widely used in the design of parameterized algorithms and also explicitly stated in [22]. It is one of the benefits of the so-called kernelization.

A data reduction rule shrinks an instance (I, k) of a parameterized problem to (I', k') , called a kernel such that $k' \leq k$ and $|I'| \leq f(k)$ for some function f . An instance is called *reduced* if any further application of reduction rule does not modify this instance. In this scenario we say that the rule has been *exhaustively applied*.

2.3 Analysis of the naive exhaustive search

The naive algorithm generates all the feasible clusterings and check their total conflicts.

Lemma 1: The time complexity of the naive exhaustive search algorithm for p -CCLI is $O(n^2 \cdot 2^{B-n})$.

Proof: For a p -partition, it is easy to see that the total conflict can be computed in $O(n^2)$ time. We need to analyze the number of feasible solutions. Since each vertex must be put into one of the clusters in its label set. Recall that $\lambda(v) = |L(v)|$. Each vertex has $\lambda(v)$ choices. Therefore, the number of feasible solutions is

$$t = \prod_{v \in V} \lambda(v).$$

Since

$$\sum_{v \in V} \lambda(v) = B,$$

we can show that $t \leq 2^{B-n}$ by induction.

When $B = n$, i.e., each vertex has only one label, $t = 1 = 2^{B-n}$. Suppose that the result holds for the total label length less than B . When the label length is B ,

$$t = \prod_{v \in V} \lambda(v) = \lambda(x) \times \prod_{v \neq x} \lambda(v),$$

in which $x = \arg \max_v \lambda(v)$ is a vertex with maximum label set. By induction hypothesis,

$$\begin{aligned} t &= \lambda(x) 2^{B-\lambda(x)-(n-1)} \\ &= 2^{B-n} \left(\frac{\lambda(x)}{2^{\lambda(x)-1}} \right) \leq 2^{B-n}. \end{aligned}$$

□

We note that the bound is tight and the extremal case happens when $\lambda(v) \leq 2$ for all $v \in V$.

3 Reduction rules and the tree-search algorithm

In this section we show the designed reduction rules and a tree-search algorithm for the p -CCLI problem.

In the tree-searching process, the final clusters of some vertices will be determined. Let X_i denote those vertices must be in cluster i and $X = \bigcup_{i=1}^p X_i$. Let $U = V \setminus X$ be the set of undetermined vertices and $\pi[U] = (U_1, U_2, \dots, U_p)$ be the current p -partition of U . For each $u \in U$, we record $c_\pi(u, X_i)$ for $i \in L(u)$ and also $\chi = c_\pi(X, X)$ which is the total conflict among the determined vertices. Let F denote the collection of $c_\pi(u, X_i)$ for $u \in U$ and $1 \leq i \leq p$. Let D denote the sequence of $|X_i|$ for $1 \leq i \leq p$ and $\mathcal{L}$ denote the collection of label sets of vertices in U . The input of the recursive procedure is $(G[U], \pi[U], \chi, D, F, \mathcal{L})$ which size is bounded by $O(|U|^2)$ since $|F|, |\mathcal{L}| \leq p|U|$ and p is a constant. We shall only focus on how to answer the decision problem or finding the minimum total conflict. The optimal p -partition can be re-constructed by back-tracking on the search tree with some additional bookkeeping. The reduction rules are listed in the follows. Initially $U = V$, $X = \emptyset$, and $\pi[U]$ is an arbitrary feasible p -partition.

- R1: For any $u \in U$ with $\lambda(u) = 1$, move u from U to X_i in which i is the element in $L(u)$. Then, preform R2.
- R2: When $u \in U_i$ is put into X_i , update $\chi \leftarrow \chi + 2c_\pi(u, X)$ and also $c_\pi(v, X_i) \leftarrow c_\pi(v, X_i) + c_\pi(u, v)$ for each $v \in U \setminus \{u\}$.
- R3: When u is moved from U_i to U_j , update $c_\pi(u, X_i) \leftarrow |X_i| - c_\pi(u, X_i)$ and $c_\pi(u, X_j) \leftarrow |X_j| - c_\pi(u, X_j)$.
- R4: When the label i is removed from $L(u)$ for $u \in U_i$, move u to U_j and perform R3 if there are still more than one label; otherwise perform R1.
- R5: If $\chi + f(\pi) > k$, return No, in which $f(\pi)$ is a lower bound defined in (1).
- R6: If $U = \emptyset$ and $\chi \leq k$, return Yes.

The same reduction rules can be also used to solve the minimization version, as we did in our experiments, with simply replacing k with the current best solution.

Below we show the correctness of the reduction rules. Rules R1, R2, R4 and R6 are trivial. Let $f_\Delta(u, i) = |X_i| - 2c_\pi(u, X_i)$. The correctness of R3 is included in the proof of the following lemma.

Lemma 2: If we move a vertex u from U_i to U_j , then $c_{\pi'}(u, X_i) = c_\pi(u, X_i) + f_\Delta(u, i)$ and $c_{\pi'}(u, X_j) = c_\pi(u, X_j) + f_\Delta(u, j)$, in which $i \neq j$ and π' is the partition obtained by moving u to U_j . Furthermore, for $r \neq i, j$, $c_{\pi'}(u, X_r) = c_\pi(u, X_r)$.

Proof: The first result follows from $c_\pi(u, X_i) = |X_i| - d(u, X_i)$ and $c_{\pi'}(u, X_i) = d(u, X_i)$. The case for j is similar. Finally, for $r \neq i, j$, the conflicts are not changed. □

For $i \neq j$, let

$$f_{ij}(\pi, u) = c_\pi(u, X) + f_\Delta(u, i) + f_\Delta(u, j).$$

Immediately,

$$c_{\pi'}(u, X) = f_{ij}(\pi, u),$$

and we have that

$$2 \sum_{i=1}^p \sum_{u \in U_i} \min \left\{ c_\pi(u, X), \min_{j \in L(u) \setminus \{i\}} f_{ij}(\pi, u) \right\} \quad (1)$$

is a correct lower-bound for the total conflict of π , and this proves the correctness of R5. The lower bound is denoted by $f(\pi)$. The time complexity for applying the rules is trivial.

Lemma 3: Each recursive call takes $O(|U|^2)$ processing time, in which U is the set of undetermined vertices.

Theorem 4: The p -CCLI problem can be solved in $O(2^{B-n} + n^2)$ time.

Proof: Let v be a vertex with maximum $\lambda(v)$ and β be the total label size $\sum_{v \in V} \lambda(v)$. Let $v \in U_i$ and $\mu = |U|$. We branch into two cases: Either (1) put v into X_i ; or (2) remove i from $L(v)$, as well as perform the according reduction rules. Let $T(\beta, \mu)$ denote the time complexity. Before the first recursive call, we take $O(n^2)$ time to exhaustively apply R1, and then, by R1 and R4, we have that $\beta \geq 2\mu$ and $\lambda(v) \geq 2$. We shall show that

$$T(\beta, \mu) \leq q_2(2^{\beta-\mu} - \mu^2) \quad (2)$$

for $\beta \geq 2\mu$ and $\mu \geq 0$, where q_2 is a constant.

When $\beta = 2\mu$, i.e., each vertex with exactly two labels, by R3 and R4, in both cases (1) and (2) we remove a vertex and its two labels. Therefore, for some constant q_1 , $T(2\mu, \mu) \leq 2T(2(\mu-1), \mu-1) + q_1\mu^2$ if $\mu > 0$ and $T(0, 0) \leq q_1$. By induction, we show that $T(2\mu, \mu) \leq q_2(2^\mu - \mu^2)$ for sufficiently large constant q_2 . When $\mu = 0$, $T(0, 0) \leq q_1 \leq q_2(2^0 - 0)$. Suppose it holds for $\mu - 1$.

$$\begin{aligned} & T(2\mu, \mu) \\ & \leq 2T(2(\mu-1), \mu-1) + q_1\mu^2 \\ & \leq 2q_2(2^{\mu-1} - (\mu-1)^2) + q_1\mu^2 \\ & = q_2(2^\mu - \mu^2) - (q_2(\mu^2 - 4\mu + 2) - q_1\mu^2) \\ & \leq q_2(2^\mu - \mu^2) \end{aligned}$$

when q_2 is sufficiently large. Therefore, (2) holds for $\beta = 2\mu$ and $\mu \geq 0$.

Next, we show that it holds for all $\beta \geq 2\mu$ by induction on $\beta - 2\mu$. Above we have shown that (2) holds for $\beta - 2\mu = 0$. For case (1), β and μ are decreased by at least three and one, respectively. For case (2), β is decreased by one. Therefore,

$$T(\beta, \mu) \leq T(\beta - 3, \mu - 1) + T(\beta - 1, \mu) + q_1\mu^2$$

if $\beta > 2\mu$. Suppose that (2) holds for $\beta - 2\mu < r$. For $\beta - 2\mu = r$, since $(\beta - 3) - 2(\mu - 1) = r - 1$, we have, by induction hypothesis,

$$T(\beta - 3, \mu - 1) \leq q_2(2^{\beta - \mu - 2} - (\mu - 1)^2).$$

Similarly, $\beta - 1 - 2\mu = r - 1$, and therefore

$$T(\beta - 1, \mu) \leq q_2(2^{\beta - \mu - 1} - \mu^2).$$

Then

$$\begin{aligned} & T(\beta, \mu) \\ & \leq q_2(2^{\beta - \mu - 2} + 2^{\beta - \mu - 1} - \mu^2) \\ & \quad - (q_2(\mu - 1)^2 - q_1\mu^2) \\ & < q_2(2^\mu - \mu^2) \end{aligned}$$

when q_2 is sufficiently large. Finally, since we take $O(n^2)$ time to perform the initial reduction, the total time complexity with input label length B is $O(2^{B-n} + n^2)$. $\square$

4 Experimental results

In this section, we show the experimental results. The experiments are for the minimization version. In experiment 1, we tested two algorithms and compared the running time.

- Algorithm 1: naive exhaustive search algorithm.
- Algorithm 2: Branch-and-bound algorithm with the trivial lower bound, i.e, stopping searching if $c_\pi(X, X)$ is at least the current best solution.

In experiment 2, we improve the branch-and-bound algorithm by using the lower bound (lb) described in the previous section. Also we compare the version with the variant which additionally use a greedy algorithm to find an initial solution. We abort the program if the number of recursive calls is more than ten millions.

4.1 Test data

The test data is built as follows. First, we built a p -cluster graph with n vertices. Then we pick one vertex as the base vertex for each cluster and randomly connect $B - n$ edges between base vertices and others. Finally, we randomly modify m edges between vertices excluding the base vertices.

4.2 Experimental results

Experiment 1

We compared the running time of Algorithms 1 and 2 for different n and B . In the following tables, we can observe that Algorithm 2 is much more efficient.

Table 1: The running time of Alg 1.

$B - n$	$n=100$	$n=300$	$n=500$
10	0.044(s)	0.389(s)	1.033(s)
15	1.200(s)	11.262(s)	32.163(s)
20	33.121(s)	>120(s)	>120(s)

Table 2: The running time of Alg 2.

$B - n$	$n=100$	$n=300$	$n=500$
10	0.010(s)	0.030(s)	0.045(s)
15	0.089(s)	0.588(s)	1.266(s)
20	1.595(s)	16.970(s)	38.446(s)

Experiment 2

In experiment 2, we set $n=500$ and observe the effect of the designed lower bound. When $B = 20$ the number of recursive calls for the one with only trivial lower bound is about 1.8 millions, and exceeds our threshold ten millions when $B \geq 25$.

In Figure 3, we compare the two versions: with or without greedy initial solution, both with the designed lower bound. The result shows that the improvement of using the designed lower bound is very significant. Furthermore, the greedy initial solution is also helpful.

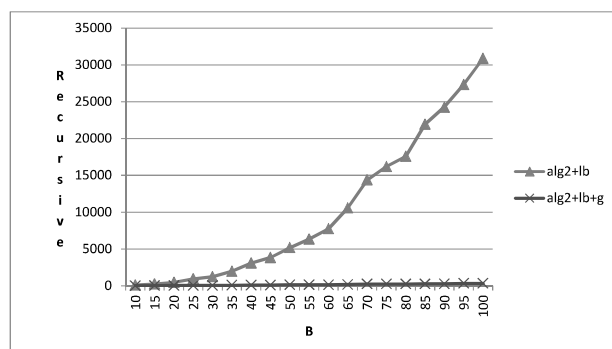


Figure 1: The number of recursive calls

5 Conclusion remarks

An important and interesting challenge is how to design a parameterized algorithm with both the label length and the total conflict as parameters. In some applications of clustering such as machine learning, there are usually only few items with labels, possibly comes from some costly methods like manual labeling. For p -CCLI can also handle items with no label. However, an empty label set is equivalent to a full label set, i.e., it can be put to any cluster. It results in an instance with too long length, and thus make the problem intractable. Therefore designing algorithm efficient for small total conflict is important. Another possible approach is to design approximation algorithms.

Acknowledgment

This work was supported in part by NSC 100-2221-E-194-036-MY3 and NSC 101-2221-E-194-025-MY3 from the National Science Council, Taiwan, R.O.C.

References

- [1] N. Ailon, M. Charikar, and A. Newman, Aggregating inconsistent information: Ranking and clustering, *J. ACM* 55(5):1–27, 2008.
- [2] N. Bansal, A. Blum, and S. Chawla, Correlation clustering, *Machine Learning, Special Issue on Clustering* 56:89–113, 2004.
- [3] S. Böcker and P. Damaschke, Even faster parameterized cluster deletion and cluster editing, *Inf. Process. Lett.* 111(14):717–721, 2011.
- [4] S. Böcker, S. Briesemeister, Q.B.A. Bui, A. Truss, Going weighted: Parameterized algorithm for cluster editing, *Theor. Comput. Sci.* 410 (52):5467–5480, 2009.
- [5] P. Bonizzoni, G. Della Vedova, and R. Dondi, A ptas for the minimum consensus clustering problem with a fixed number of clusters, in: *Proc. Eleventh Italian Conference on Theoretical Computer Science*, 2009.
- [6] P. Bonizzoni, G. Della Vedova, R. Dondi, and T. Jiang, On the approximation of correlation clustering and consensus clustering, *J. Comput. System Sci.* 74(5): 671–696, 2008.
- [7] J. Chen, J. Meng, A $2k$ kernel for the cluster editing problem, in: M.T. Thai, S. Sahni (Eds.), *Computing and Combinatorics Conf., COCOON 2010*, in: LNCS, vol. 6196, 2010, pp. 459–468.
- [8] L.-H. Chen, M.-S. Chang, C.-C. Wang, B. Y. Wu, On the Min-Max 2-Cluster Editing Problem, in R.-S. Chang, L. C. Jain, S.-L. Peng (Eds.), *Advances in Intelligent Systems and Applications, Vol. 1*, in: *Smart Innovation, Systems and Technologies*, vol. 20, 2013, pp. 133–142.
- [9] P. Damaschke, Bounded-degree techniques accelerate some parameterized graph algorithms, in: J. Chen, F.V. Fomin (Eds.), *Int. Workshop on Parameterized and Exact Comp., IWPEC 2009*, in: LNCS, vol. 5917, 2009, pp. 98–109.
- [10] P. Damaschke, Fixed-parameter enumerability of cluster editing and related problems, *Theory Computing Syst.* 46:261–283, 2010.
- [11] R. G. Downey and M. R. Fellows, *Parameterized Complexity*, Springer-Verlag, New York, 1999.
- [12] M. R. Fellows, J. Guo, C. Komusiewicz, R. Niedermeier, and J. Uhlmann, Graph-based data clustering with overlaps, *Discrete Optim.* 8(1):2–17, 2011.

- [13] V. Filkov and S. Skiena, Integrating microarray data by consensus clustering, *International Journal on Artificial Intelligence Tools* 13(4): 863–880, 2004.
- [14] F. Fomin, S. Kratsch, M. Pilipczuk, M. Pilipczuk and Y. Villanger, Tight bounds for parameterized complexity of cluster editing, to appear in *STACS 2013*, also available at <http://arxiv.org/abs/1112.4419>
- [15] I. Giotis and V. Guruswami, Correlation clustering with a fixed number of clusters, *Theory Comput.* 2:249–266, 2006.
- [16] J. Gramm, J. Guo, F. Hüffner, R. Niedermeier, Graph-modeled data clustering: Fixed parameter algorithms for clique generation, *Theory Computing Syst.* 38:373–392, 2005.
- [17] J. Gramm, J. Guo, F. Hüffner, R. Niedermeier, Automated generation of search tree algorithms for hard graph modification problems, *Algorithmica* 39:321–347, 2004.
- [18] J. Guo, A more effective linear kernelization for cluster editing, *Theor. Comput. Sci.* 410:718–726, 2009.
- [19] F. Harary, On the notion of balance of a signed graph, *Michigan Mathematical Journal* 2(2):143–146, 1953.
- [20] F. Hüffner, C. Komusiewicz, H. Moser, and R. Niedermeier, Fixed-parameter algorithms for cluster vertex deletion, *Theory Computing Syst.* 47(1):196–217, 2010.
- [21] C. Komusiewicz and J. Uhlmann. Cluster editing with locally bounded modifications, *Discret. Appl. Math.*, 160(15):2259–2270, 2012.
- [22] R. Niedermeier and P. Rossmanith, A general method to speed up fixed-parameter tractable algorithms, *Inform. Process. Lett.* 73 (2000), 125–129.
- [23] R. Shamir, R. Sharan, and D. Tsur, Cluster graph modification problems, *Discr. Appl. Math.* 144(1–2):173–182, 2004.
- [24] S. Wasserman and K. Faust, *Social Network Analysis*, Cambridge University Press, Cambridge, 1994.